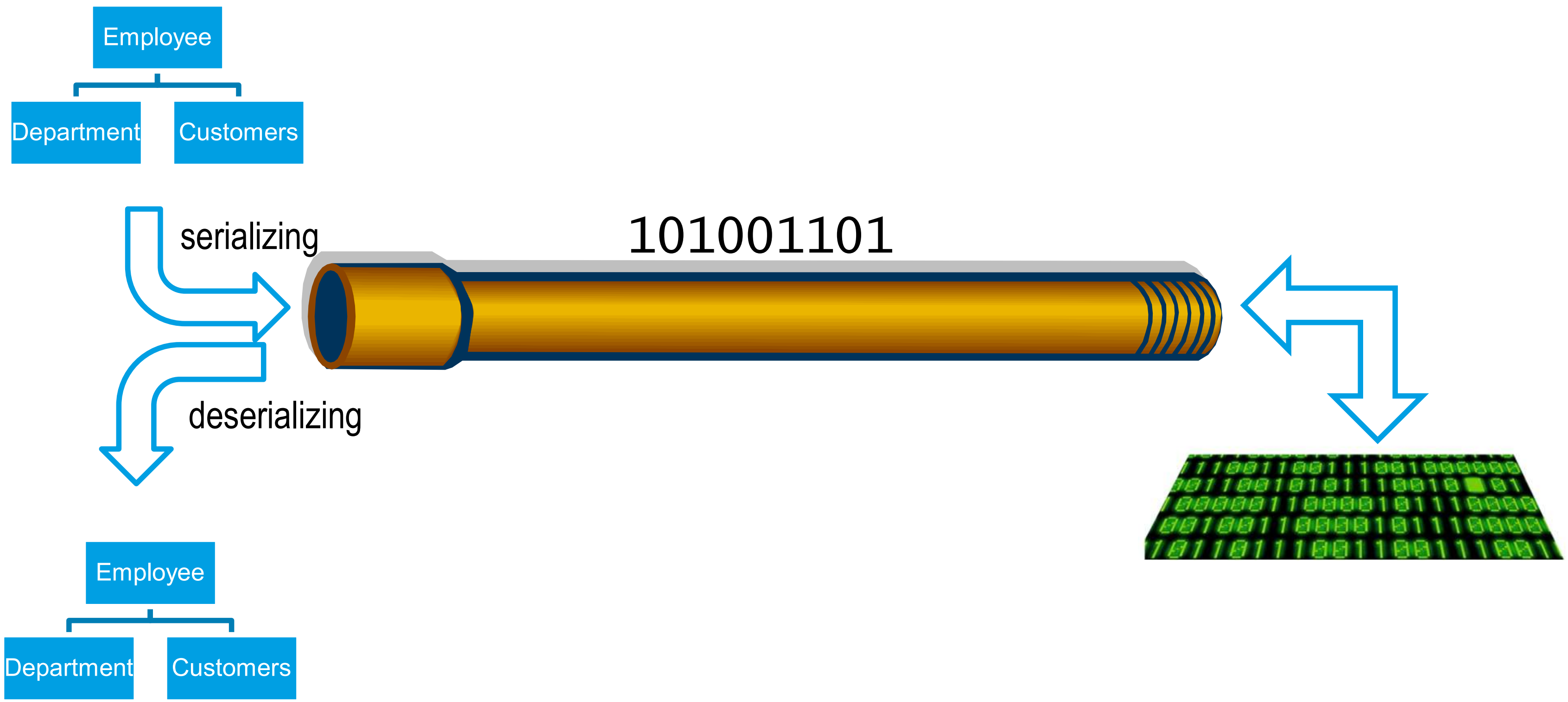
Module 13: Serialization and Deserialization

Overview

- Serialization and Deserialization
- Deep and Shallow Serialization
- XML Serialization
- Binary Serialization
- JSON Serialization

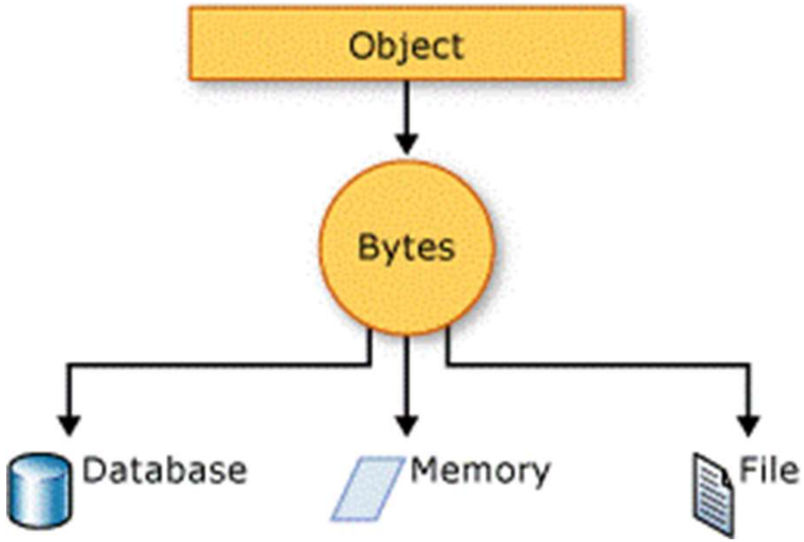
Serialization and Deserialization

- process of converting the state of an object into a form that can be persisted or transported.



Benefits of Serialization and Deserialization

- Object recreation
- Storage of objects
- Data exchange



Deep and Shallow Serialization

- Deep serialization
  - serializes the whole object and any objects referenced by the object being serialized (Binary, Soap)

- Shallow serialization
  - allows for the serialization of public read-write property values (Web Services and XmlSerializer)

XML Serialization

- Shallow serialization
- Serializes into a XML Format
  - Easily readable and editable
  - Cross platforms data interchange
- Attributes can be applied to control the way the XmlSerializer serializes

```
using System.Xml.Serialization;
```

XmlSerializer Class

- Serializes and deserializes objects into and from XML

- Serialization

```
XmlSerializer xmlSerializer = new XmlSerializer(typeof(MyClass));
xmlSerializer.Serialize(myStream, myObject);
```

- Deserialization

```
MyClass myDeserializedObject =
        (MyClass)serializer.Deserialize(myStream);
```

Attributes to control XML serialization

- XmlElement
  - Controls how the XmlSerializer serializes or deserializes an object

```
[XmlElement(
   ElementName = "FName",
   Namespace = "http://www.gopas.cz")]
   public string FirstName;
```

- XmlIgnore
  - *Instructs XmlSerializer not to serialize property value*

Note:

- *Other attributes XmlAttribute, XmlRoot, XmlText, etc.*

Restrictions of XML Serialization

- Only public read-write members are serialized
- There must be public default contructor in class being serialized

Lab 13.1 - XML Serialization

Exercise:

1. Creating class
2. XML Serialization
3. XML Deserialization

Binary Serialization

- Serializes into an BinaryFormat
- Deep serialization/deserialization
- Serializing into Binary format - not readable not editable format

```
using System.Runtime.Serialization;
```

- Advantages
  - performance
  - deep serialization
  - brief

BinaryFormatter Class

- Serializes and deserializes an object, or an entire graph of connected objects, in binary format.

- Serialization

```
IFormatter formatter = new BinaryFormatter();
formatter.Serialize(myStream, myObject);
```

- Deserialization

```
IFormatter formatter = new BinaryFormatter();
myObject = (MyClass)formatter.Deserialize(myStream);
```

SerializableAttribute Class

- Indicates that a class can be Serialized
- **SerializationException** if any type in the graph of objects being serialized does not have the **SerializableAttribute** attribute
- **NonSerializedAttribute** attribute excludes fields from serialization

```
[Serializable()]
class MyClass
{
}
```

IDeserializationCallback Interface

- Class will be notified when deserialization of the entire object graph has been completed.

```
void IDeserializationCallback.OnDeserialization(Object sender)
{
}
```

Note:

- *The functionality for **sender** parameter is not currently implemented.*

Lab 13.2 - Binary Serialization

Exercise:

1. Creating class
2. Binary Serialization
3. Binary Deserialization

JSON Serialization

- Lightweight data-interchange format
- Easy for humans to read and write

```
{"FirstName":"Karel", "LastName": "Novák"}
```

- It is easy for machines to parse and generate
- It is based on a subset of the JavaScript Programming Language
  a) System.Web.Script.Serialization
  b) Newtonsoft.Json
  c) System.Text.Json

Lab 13.3 - JSON

Exercise:

1. JSON Serialization

Review

- Serialization and Deserialization
- Deep and Shallow Serialization
- XML Serialization
- Binary Serialization
- JSON Serialization